

Phase 6: User Interface Development

Doctor Appointment & Consultation System (Healthcare CRM)

Objective

The goal of this phase is to enhance the user experience of the *Doctor Appointment & Consultation System* by creating customized Lightning pages, apps, and simple LWC components to make the system more interactive and user-friendly for doctors, patients, and administrators.

Components Implemented

1. Lightning App Builder

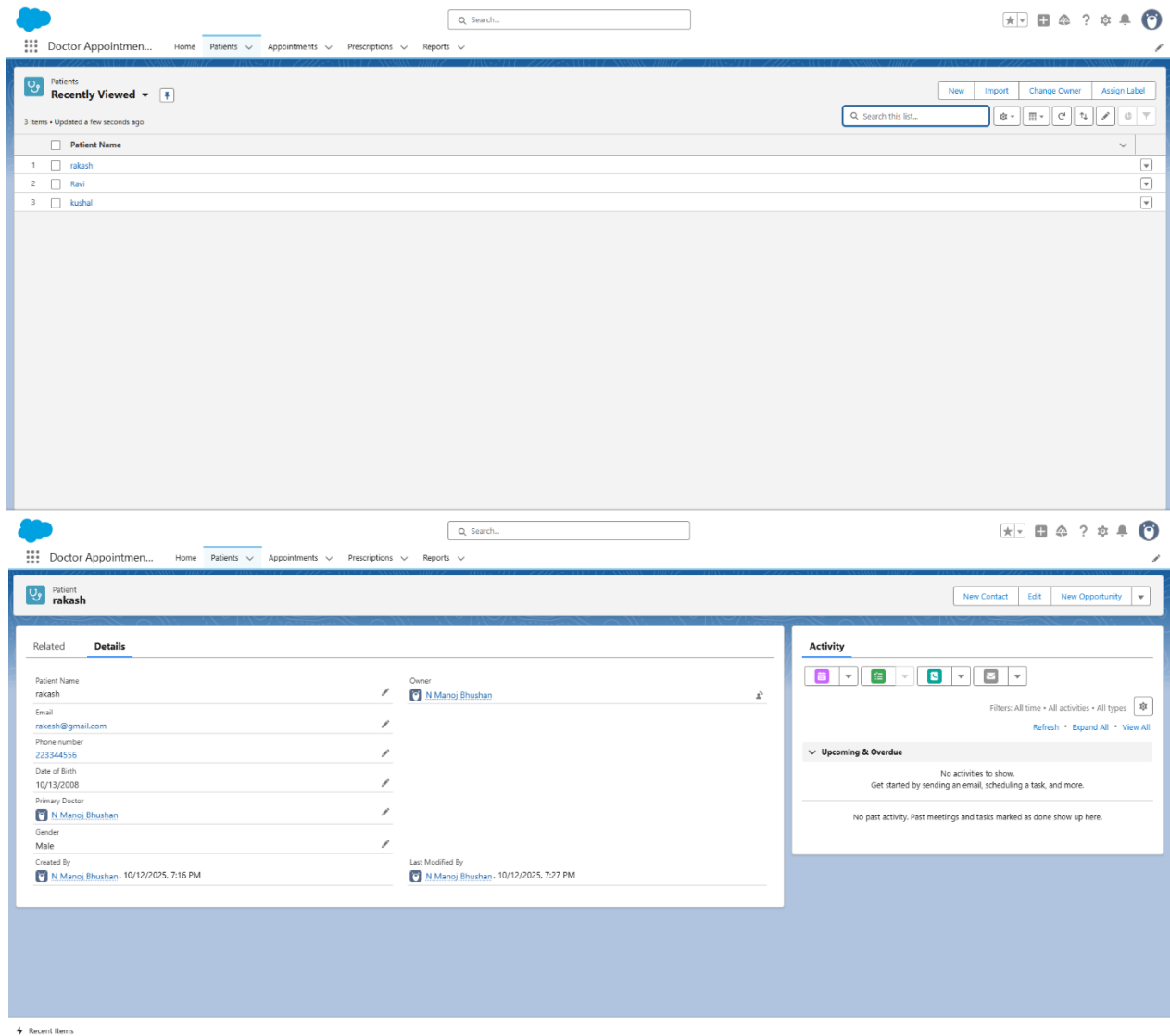
- A custom Lightning App named “**Doctor Appointment CRM**” was created using the **App Manager**.
- The app contains navigation tabs for:
 - **Patients**
 - **Appointments**
 - **Prescriptions**
 - **Reports**
- App branding was customized with the logo and color theme relevant to the healthcare domain.

The screenshot displays the Lightning App Builder interface for configuring the 'Doctor Appointment CRM' app. The top navigation bar includes 'Lightning App Builder', 'App Settings', 'Pages', and 'Doctor Appointment CRM'. The left sidebar shows 'App Settings' with 'App Details & Branding' selected. The main content area is titled 'App Details & Branding' and includes instructions: 'Give your Lightning app a name and description. Upload an image and choose the highlight color for its navigation bar.' The configuration is divided into two columns: 'App Details' and 'App Branding'. Under 'App Details', there are fields for 'App Name' (filled with 'Doctor Appointment CRM'), 'Developer Name' (filled with 'Doctor_Appointment_CRM'), and 'Description' (filled with 'Doctor Appointment & Consultation Management'). Under 'App Branding', there is an 'Image' upload section with an 'Upload' button, and a 'Primary Color Hex Value' field (filled with '#7EC2EA'). Below these, there are 'Org Theme Options' with a checkbox 'Use the app's image and color instead of the org's custom theme' (which is unchecked). At the bottom, an 'App Launcher Preview' shows a blue square icon with 'DA' and the app name 'Doctor Appointment CRM'.

2. Record Pages

- Custom Record Pages were designed for **Patient** and **Appointment** objects using **Lightning App Builder**.
- Key highlights:

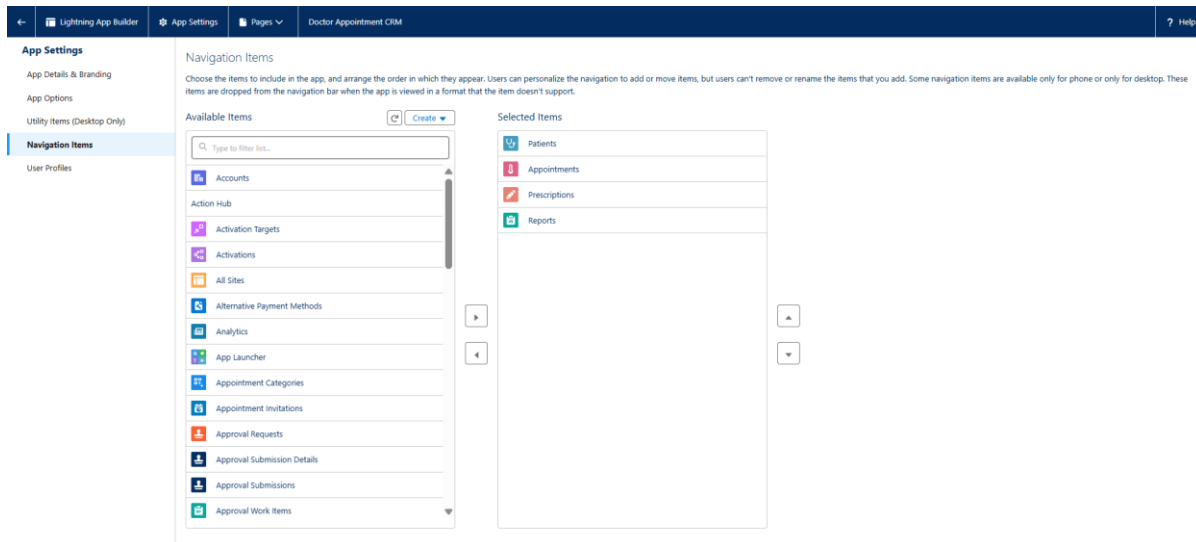
- Added **Highlights Panel** to show key details (Patient Name, Doctor Name, Appointment Date, Status).
- Included **Related Lists** for viewing associated Prescriptions and Appointments.
- Used **Dynamic Components Visibility** to show fields like *Prescription Details* only when the status = “Completed”.



3. Tabs

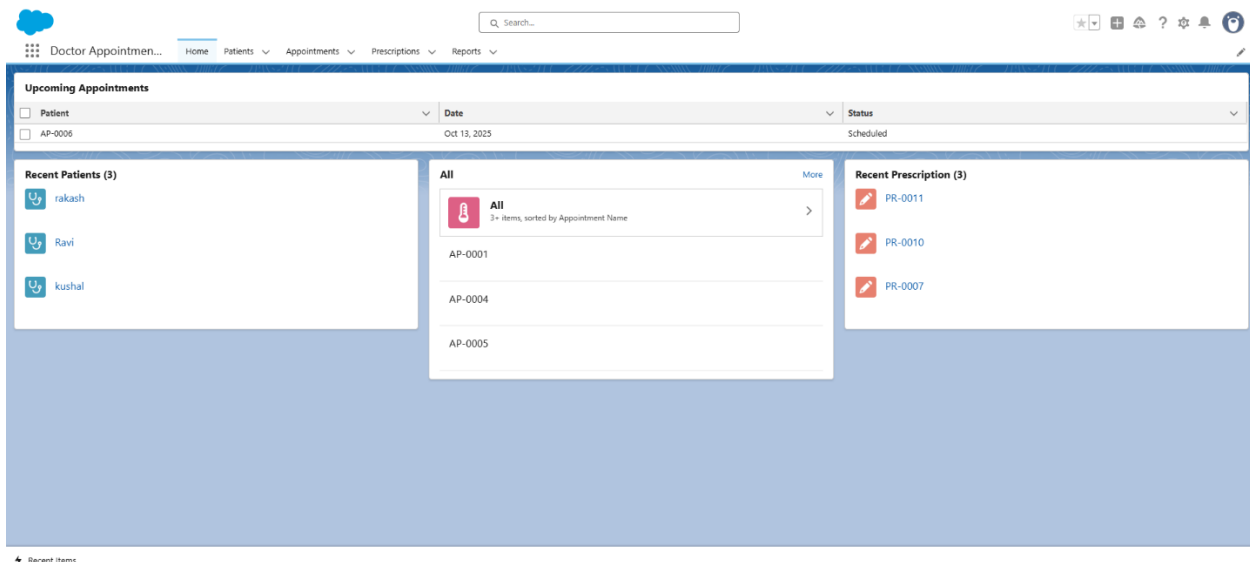
- Created custom tabs for the following custom objects:
 - **Patient__c**
 - **Appointment__c**
 - **Prescription__c**
- These tabs were added to the **Doctor Appointment CRM** app for easy navigation.

- Visibility of tabs configured so Doctors can only see relevant sections.



4. Home Page Layout

- Customized the **Home Page** for the app using Lightning App Builder.
- Added standard components:
 - **Recent Items** – to access recently viewed records.
 - **Dashboard Snapshot** – to show appointment summary and daily schedule.
 - **List View Chart** – displaying count of completed and pending appointments.
- Personalized Home Page made the system dashboard-like for daily use.



5. Utility Bar

- Added a **Utility Bar** to the bottom of the Lightning App for quick access:
 - **“Create New Appointment”** – opens a quick form to add an appointment.
 - **“Help & Support”** – shows FAQ and contact details.
 - **“Notes”** – for doctors to write quick notes.

6. Lightning Web Component (LWC)

A simple **LWC component** was developed to display a doctor’s upcoming appointments dynamically.

Component Name: doctorUpcomingAppointments

Features:

- Uses **Apex Class (DoctorUtilityV2)** to fetch appointments.
- Displays upcoming appointments in a data table.
- Refreshes automatically when new appointments are created.

Code Snippet (Simplified):

```
// doctorUpcomingAppointments.js
```

```
import { LightningElement, wire } from 'lwc';
```

```
import getDoctorAppointments from '@salesforce/apex/DoctorUtilityV2.getDoctorAppointments';
```

```
export default class DoctorUpcomingAppointments extends LightningElement {
```

```
  appointments;
```

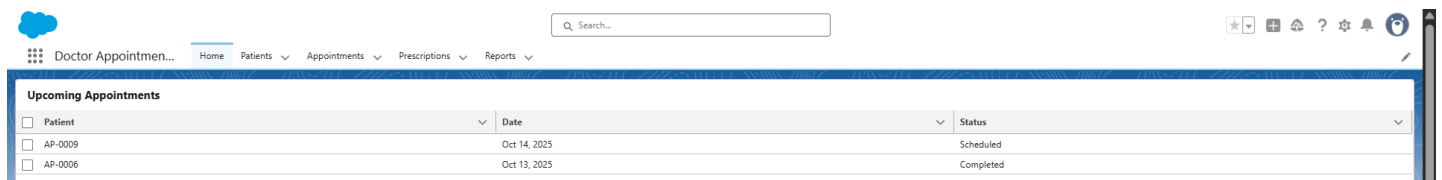
```
  @wire(getDoctorAppointments, { doctorId: '$userId' })
```

```
  wiredAppointments({ data, error }) {
```

```
    if (data) this.appointments = data;
```

```
  }
```

```
}
```



The screenshot shows a Salesforce Lightning App interface. At the top, there is a navigation bar with a search bar and several icons. Below the navigation bar, there is a table titled "Upcoming Appointments". The table has three columns: "Patient", "Date", and "Status". There are two rows of data in the table.

Patient	Date	Status
AP-0009	Oct 14, 2025	Scheduled
AP-0006	Oct 13, 2025	Completed

7. Apex with LWC

- The **LWC** used Apex method `getDoctorAppointments` from `DoctorUtilityV2` class.
- The method fetches Appointment records filtered by **Doctor__c** and sorts them by date.

8. Events in LWC

- Implemented simple event handling:
 - When user clicks on an appointment record, a **custom event** navigates to the record page using the Navigation Service.

9. Wire Adapters & Imperative Apex

- Used **@wire adapter** to get data reactively from Apex.
- Tested **imperative Apex calls** to refresh appointment data manually when status changes.

10. Navigation Service

- Implemented Salesforce **NavigationMixin** in LWC to allow redirection to record pages directly.

Example:

```
import { NavigationMixin } from 'lightning/navigation';

this[NavigationMixin.Navigate]({
  type: 'standard__recordPage',
  attributes: {
    recordId: record.Id,
    actionName: 'view'
  }
});
```